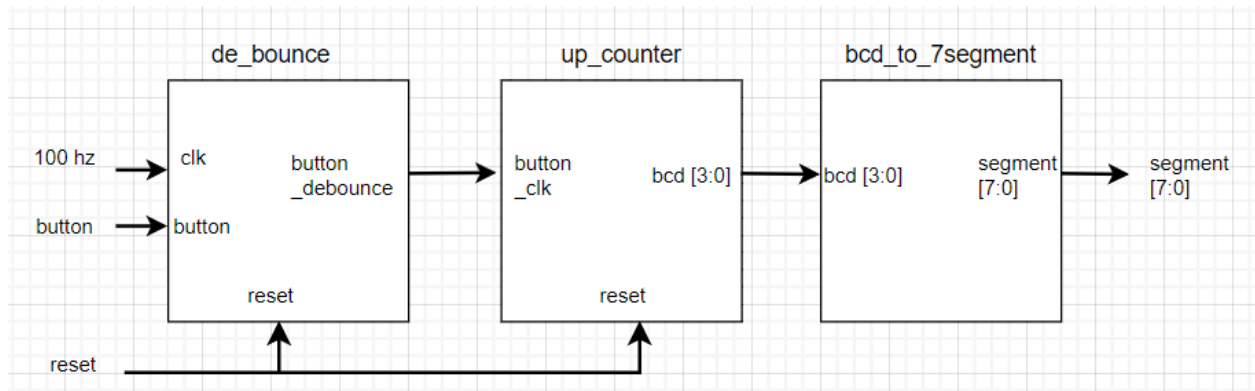


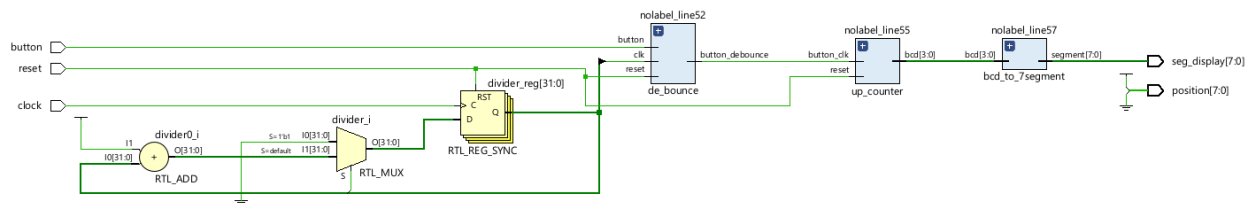
Digital Logic Design – Lab 5

Task 1

1. **Topic:** 0 to 9 up-counter controlled by a push button
2. **Design structure:**



Also, this is the circuit block diagram generated by Vivado from my source code



3. **Code:** the code for task 1 is in folder Task_1

- **de_bounce.v**

```
22 // implementation of the debounce circuit for push button
23 module de_bounce(clk, button, reset, button_debounce);
24     input clk, button, reset;
25     output button_debounce;
26
27     wire [2:0] q_out;
28
29     dff dff_1 (button, clk, reset, q_out[0]);
30     dff dff_2 (q_out[0], clk, reset, q_out[1]);
31     dff dff_3 (q_out[1], clk, reset, q_out[2]);
32
33     assign button_debounce = q_out[0] & q_out[1] & q_out[2];
34 endmodule
```

```

36 | // basic d flip-flop used in the de-bounce circuit
37 | module dff(d, c, rst, q);
38 |     input d, c, rst;
39 |     output reg q;
40 |
41 |     always @(posedge c) begin
42 |         if(rst) // when reset pressed, q becomes 0
43 |             q <= 1'b0;
44 |         else
45 |             q <= d;
46 |     end
47 | endmodule

```

- up_counter.v

```

22 | // up counter that goes from 0 to 9 with every push of the button
23 | module up_counter(button_clk, reset, bcd);
24 |     input button_clk, reset;
25 |     output reg [3:0] bcd; // count 0~9
26 |
27 |     always @(posedge button_clk or posedge reset) begin
28 |         if(reset) // when reset pushed, bcd is 0
29 |             bcd <= 4'b0000;
30 |         else if(bcd >= 9) // when bcd exceeds 9, set bcd to 0
31 |             bcd <= 4'b0000;
32 |         else // with every button pushed, increase bcd by 1
33 |             bcd <= bcd + 1;
34 |     end
35 | endmodule

```

- bcd_to_7segment.v

```

22 // use the bcd count to display the appropriate 7 segment LEDs
23 module bcd_to_7segment(bcd, segment);
24     input [3:0] bcd;
25     output reg [7:0] segment;
26
27     always @(bcd) begin
28         case(bcd)
29             4'b0000 : segment <= 8'b11111100; // 0
30             4'b0001 : segment <= 8'b01100000; // 1
31             4'b0010 : segment <= 8'b11011010; // 2
32             4'b0011 : segment <= 8'b11110010; // 3
33             4'b0100 : segment <= 8'b01100110; // 4
34             4'b0101 : segment <= 8'b10110110; // 5
35             4'b0110 : segment <= 8'b10111110; // 6
36             4'b0111 : segment <= 8'b11100000; // 7
37             4'b1000 : segment <= 8'b11111110; // 8
38             4'b1001 : segment <= 8'b11110110; // 9
39             default : segment <= 8'b01101110; // H
40         endcase
41     end
42 endmodule

```

- up_counter_0to9.v

```

22 // this is the final module, this module combines all previous modules:
23 // de_bounce(), up_counter(), and bcd_to_7segment(). That way, when
24 // you push the button, the correct number is displayed
25 module up_counter_0to9(clock, button, reset, seg_display, position);
26     input clock, button, reset;
27     output [7:0] seg_display;
28     output [7:0] position;
29
30     assign position = 8'b00000001; // position of LED being displayed
31
32     // Clock divider
33     // 100 Hz approx. to div[20]=10.4ms
34     reg [31:0] divider;
35
36     always @(posedge clock) begin
37         if(reset)
38             divider <= 0;
39         else if(divider[23] == 1'b1)
40             divider <= 0;
41         else
42             divider <= divider + 1;
43     end

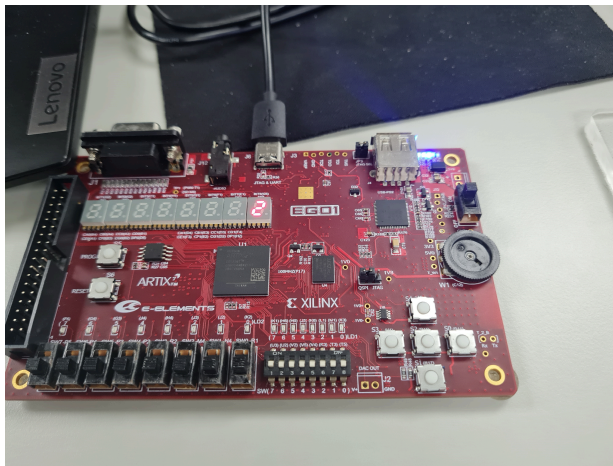
```

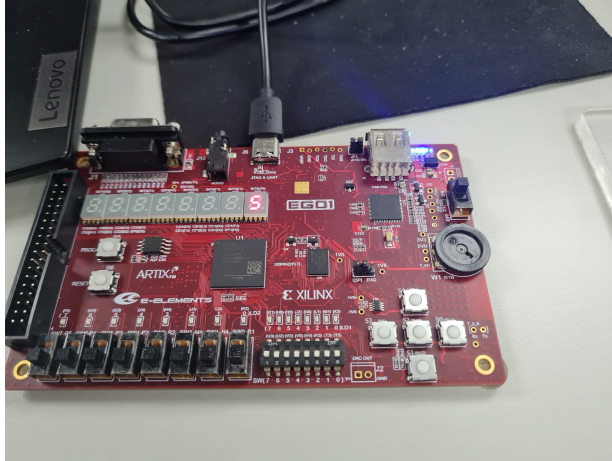
```

45 // Remember that...
46 //
47 // module de_bounce(clk, button, reset, button_debounce);
48 // module up_counter(button_clk, reset, bcd);
49 // module bcd_to_7segment(bcd, segment);
50
51 wire button_clk;
52 de_bounce(divider[20], button, reset, button_clk);
53
54 wire [3:0] bcd;
55 up_counter(button_clk, reset, bcd);
56
57 bcd_to_7segment(bcd, seg_display);
58
59 endmodule

```

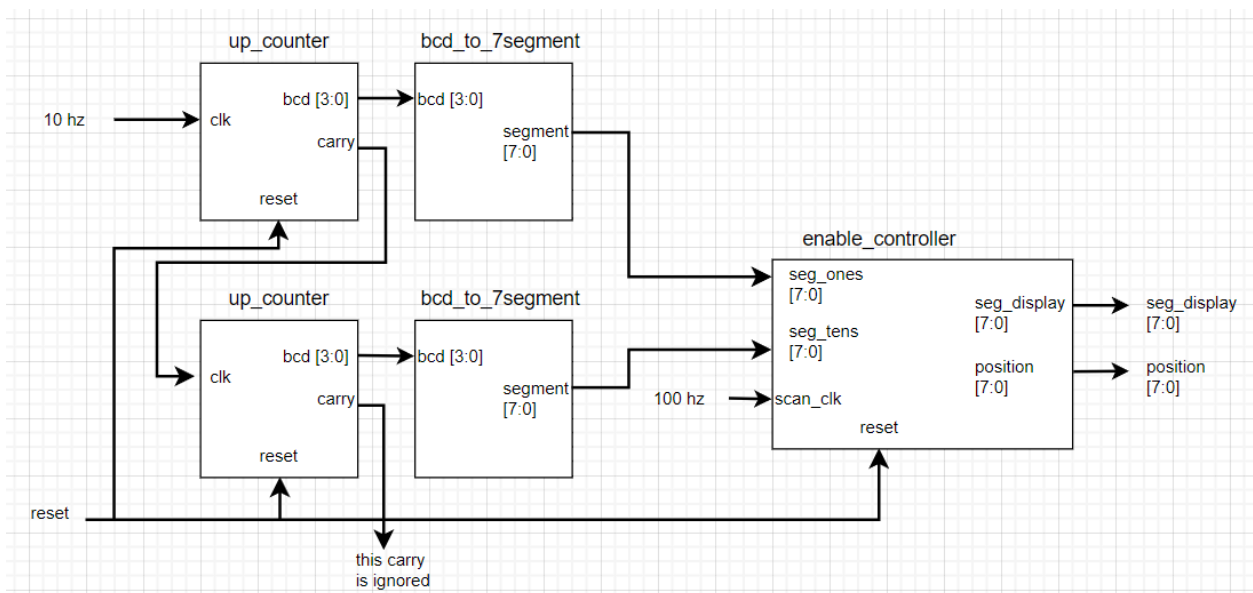
4. **Result:** the working of the FPGA board can be seen on the following link https://www.youtube.com/watch?v=kj_QjdsJIQE. When the button on pin U4 is pressed, the number increases, the number goes from 0 to 9, and when the current number displayed is 9, the next number to be displayed is 0 again. When the reset button on pin R17 is pressed, the number is set to 0



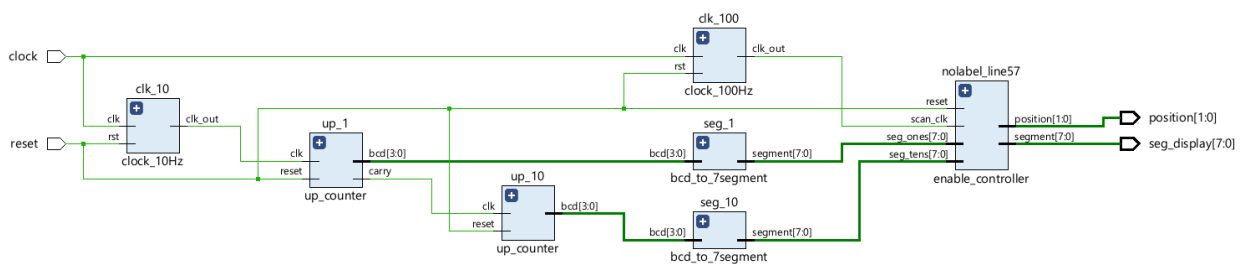


Task 2

1. Topic: 00 to 99 automatic up-counter
2. Design structure:



Also, this is the circuit block diagram generated by Vivado from my source code



3. **Code:** the code is in the folder Task_2. The whole project is implemented in up_counter_00to99.v, which uses the different modules in enable_controller.v, up_counter.v, and bcd_to_7segment.v

- **enable_controller.v**

```
22 | // this module allows the display of 2 different LEDs at the same time. This is done
23 | // by lighting up one LED, and then the other one, at fast velocities, faster than the
24 | // naked eye can process, so we see as if 2 LEDs are light up at the same time
25 | module enable_controller(scan_clk, reset, seg_tens, seg_ones, segment, position);
26 |     input scan_clk, reset;
27 |     input [7:0] seg_tens, seg_ones;
28 |
29 |     output reg [7:0] segment;
30 |     output reg [1:0] position;
31 |
32 |     reg counter;
33 |     always @(posedge scan_clk or posedge reset) begin
34 |         if(reset) begin
35 |             counter <= 1'b0;
36 |             position <= 2'b01;
37 |             segment <= seg_ones;
38 |         end
39 |         else begin
40 |             if(counter == 1'b0) begin
41 |                 position <= 2'b01;
42 |                 segment <= seg_ones;
43 |             end
44 |
45 |             else begin
46 |                 position <= 2'b10;
47 |                 segment <= seg_tens;
48 |             end
49 |
50 |             counter <= ~counter;
51 |         end
52 |     end
53 | endmodule
```

```

56 // The clock output is defined by the counter set to divide the
57 // clock of 100 MHz. Dividing clock of 100 MHz by 1.000.000 gives
58 // a 100 Hz clock
59 module clock_100Hz(clk, rst, clk_out);
60     input clk, rst;
61     output reg clk_out;
62
63     integer counter;
64     always @(posedge clk or posedge rst) begin
65         if(rst)
66             counter <= 0;
67         else if(counter == 1000000) // divide clock by 1.000.000
68             counter <= 0;
69         else
70             counter <= counter + 1;
71     end
72
73     // create clock signal, the 1st part of 1.000.000 should be 1,
74     // and the 2nd part should be 0
75     always @(posedge clk or posedge rst) begin
76         if(rst)
77             clk_out <= 1'b0;
78         else if(counter <= 500000) // 1st half of 1.000.000
79             clk_out <= 1'b1;
80         else
81             clk_out <= 1'b0;
82     end
83 endmodule
84
85 // The clock output is defined by the counter set to divide the
86 // clock of 100 MHz. Dividing clock of 100 MHz by 10.000.000 gives
87 // a 10 Hz clock
88 module clock_10Hz(clk, rst, clk_out);
89     input clk, rst;
90     output reg clk_out;
91
92     integer counter;
93     always @(posedge clk or posedge rst) begin
94         if(rst)
95             counter <= 0;
96         else if(counter == 10000000) // divide clock by 10.000.000
97             counter <= 0;
98         else
99             counter <= counter + 1;
100    end

```

```

102 // create clock signal, the 1st part of 10.000.000 should be 1,
103 // and the 2nd part should be 0
104 always @(posedge clk or posedge rst) begin
105     if(rst)
106         clk_out <= 1'b0;
107     else if(counter <= 5000000) // 1st half of 10.000.000
108         clk_out <= 1'b1;
109     else
110         clk_out <= 1'b0;
111 end
112 endmodule

```

- up_counter.v

```

22 // up counter that goes from 0 to 9 automatically because of the clock
23 module up_counter(clk, reset, bcd, carry);
24     input clk, reset;
25     output reg [3:0] bcd; // count 0~9
26     output reg carry; // to carry to tens
27
28     always @(posedge clk or posedge reset) begin
29         if(reset) begin
30             carry <= 1'b0;
31             bcd <= 4'b0000;
32         end
33         else if(bcd >= 9) begin
34             carry <= 1'b1;
35             bcd <= 4'b0000;
36         end
37         else begin
38             carry <= 1'b0;
39             bcd <= bcd + 1;
40         end
41     end
42 endmodule

```

- bcd_to_7segment.v

```

22 // use the bcd count to display the appropriate 7 segment LEDs
23 module bcd_to_7segment(bcd, segment);
24     input [3:0] bcd;
25     output reg [7:0] segment;
26
27     always @(bcd) begin
28         case(bcd)
29             4'b0000 : segment <= 8'b11111100; // 0
30             4'b0001 : segment <= 8'b01100000; // 1
31             4'b0010 : segment <= 8'b11011010; // 2
32             4'b0011 : segment <= 8'b11110010; // 3
33             4'b0100 : segment <= 8'b01100110; // 4
34             4'b0101 : segment <= 8'b10110110; // 5
35             4'b0110 : segment <= 8'b10111110; // 6
36             4'b0111 : segment <= 8'b11100000; // 7
37             4'b1000 : segment <= 8'b11111110; // 8
38             4'b1001 : segment <= 8'b11110110; // 9
39             default : segment <= 8'b01101110; // H
40         endcase
41     end
42 endmodule

```

- up_counter_00to99.v

```

22 // this is the final module, this module combines all previous modules:
23 // enable_controller(), up_counter(), and bcd_to_7segment(). That way,
24 // the counter goes from 00 to 99 automatically
25 module up_counter_00to99(clock, reset, seg_display, position);
26     input clock, reset;
27     output [7:0] seg_display;
28     output [1:0] position;
29
30     // Initializing clocks
31     //
32     // module clock_100Hz(clk, rst, clk_out);
33     // module clock_10Hz(clk, rst, clk_out);
34
35     wire clk_100_out, clk_10_out;
36     clock_100Hz clk_100(clock, reset, clk_100_out);
37     clock_10Hz clk_10(clock, reset, clk_10_out);

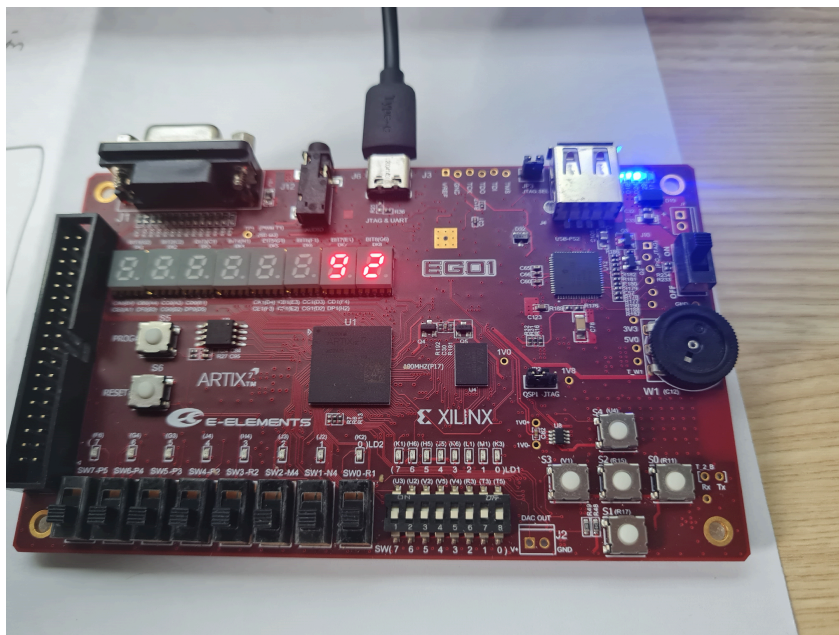
```

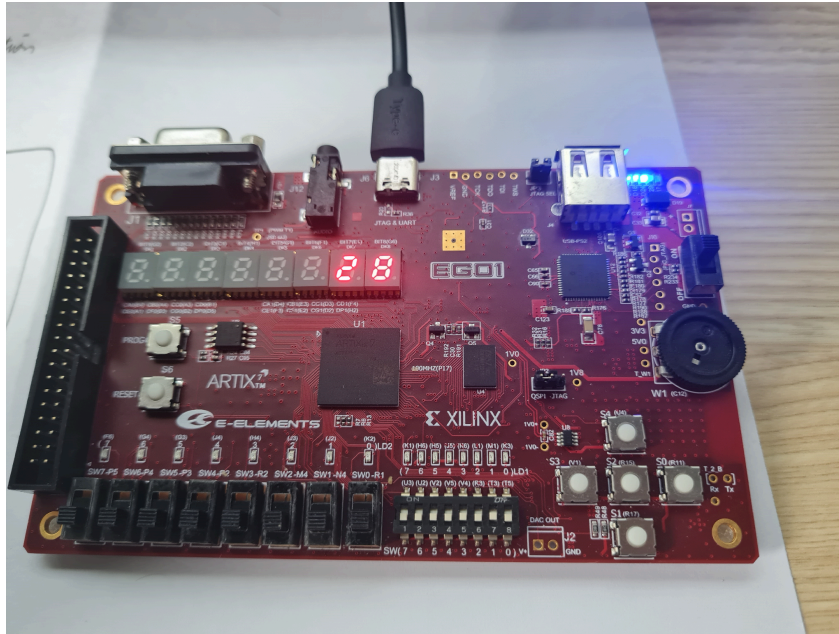
```

39 // Remember that...
40 //
41 // module enable_controller(scan_clk, reset, seg_tens, seg_ones, segment, position);
42 // module up_counter(clk, reset, bcd, carry);
43 // module bcd_to_7segment(bcd, segment);
44
45 wire [3:0] bcd_ones, bcd_tens;
46 wire [1:0] carry;
47 wire [7:0] seg_ones, seg_tens;
48
49 // ones
50 up_counter up_1(clk_10_out, reset, bcd_ones, carry[0]);
51 bcd_to_7segment seg_1(bcd_ones, seg_ones);
52
53 // tens
54 up_counter up_10(carry[0], reset, bcd_tens, carry[1]);
55 bcd_to_7segment seg_10(bcd_tens, seg_tens);
56
57 enable_controller(clk_100_out, reset, seg_tens, seg_ones, seg_display, position);
58
59 endmodule

```

4. **Result:** the demo of the code can be seen on the following link <https://www.youtube.com/watch?v=9RUUQ9WVov0>. The counter is automatic, it goes from 00 to 99 on its own. When it reaches 99 it loops back to 00 and continues. Whenever you press the reset button on pin V1, the counter is set to 00. Although it appears to display 2 LEDs at the same time, the FPGA board can only display 1 LED at a time. So, the LEDs are displayed one at a time but change in a frequency of 100 Hz, and thus, giving the illusion of 2 LEDs at the same time





5. Reflection

In this lab, we worked on up counters from 0 to 9, and 00 to 99. To do this, we had to learn how to work with clocks at different frequencies. For the 00~99 counter, we had to implement a 10 Hz clock to control the up counting, and another 100 Hz clock to switch which LED is currently being display. The 100 Hz clock switches so fast it is imperceptible to the naked eye, thus generating the illusion of 2 LEDs being light up at the same time